

COMP3000 Week 1

Team Based Learning + Domain-Specific Languages

COMP3000 Programming Languages



Welcome to COMP3000

- ▶ 13 lectures, 110 minutes each
- ▶ Every lecture is followed by a two-hour collaborative workshop
- ▶ Today: what the unit is about, how classes run, and the big idea that drives everything
- ▶ No preparation needed this week – but from Week 2 on there is



What the unit is about

- ▶ You will **design and build your own programming language**
- ▶ Not a toy: a language that solves a real problem
- ▶ The domain this year: **modelling water flows in river systems**



The Googong Dam – one of the river flows students model.

Why a language for water flows?

- ▶ River managers need to know what rainfall does to river levels over time
- ▶ Flood prediction, dam strategies, public safety
- ▶ Normally done with simulation software
- ▶ We will do it with a **custom language** instead
- ▶ No such language exists yet – you will build it

How the classes run – Team Based Learning

- ▶ The unit runs on **TBL**: you work in a team for all 13 weeks
- ▶ Each week follows a fixed cycle:
 - ▶ **iRAT** – individual multiple-choice quiz (done before class)
 - ▶ **tRAT** – the same quiz, answered with your team
 - ▶ **FAT** – class-wide discussion of the answers
 - ▶ **Application Task** – one hour of team work on the assignment
- ▶ Pre-reading every week, starting Week 2

Teams matter

- ▶ You'll form teams today and stay in them all semester
- ▶ The workshops build your assignment *as a team*
- ▶ Everyone draws on the shared pool of team work for their own submission
- ▶ Teams write a **contract** in Week 1: expectations, and how a member can be excluded
- ▶ If you stop showing up, your team can replace you

Domain-specific languages – the big idea

- ▶ A **domain-specific language** (DSL) is a language built for one job
- ▶ Ch 1 calls them “*little languages*” – “pidgins tailor-built to a specific task”
- ▶ Think application scripting languages, template engines, markup formats, config files
- ▶ Contrast with a **general-purpose language** (GPL): Java, C, Python – built for any job



Little languages are everywhere

Make XSLT Inform YAML ANTLR CFML SWIG
IDL Emacs Lisp Jinja cpp INI JSON Vim Script
Lex Sed Bash Batch Mustache AWK CSS HTML
Scsh XML yacc SQL Tcl Bison Guile ASP.NET

- ▶ SQL, regex, LaTeX, Make, CSS, shell – all little languages
- ▶ Almost every large software project needs a handful
- ▶ When you can, **reuse an existing one** instead of rolling your own

Some well-known DSLs

- ▶ **SQL** – querying relational databases
- ▶ **Regular expressions** – pattern matching in text
- ▶ **LaTeX** – typesetting documents
- ▶ **Make** – describing build dependencies
- ▶ **Dream Maker** – the game *Space Station 13* is built in it
- ▶ **strudel.cc** – making music by writing code in the browser

Why build a DSL at all?

- ▶ When a general-purpose language is **too much**:
 - ▶ too verbose for the job
 - ▶ too hard for the people who need to write it
- ▶ A DSL says *exactly* what the task needs, in the task's own terms
- ▶ The classic trade-off from ch 1: reuse an existing one... or build your own when nothing fits

Languages are great exercise

- ▶ Implementing a language is a *real* test of programming skill
- ▶ Recursion, trees, graphs, dynamic arrays, hash tables – all from scratch
- ▶ “Rise to it, and you’ll come away a stronger programmer”

There's no magic

- ▶ Languages feel magical – until you build one
- ▶ “There is no magic at all. It’s just code, and the people who hack on languages are just people.” – ch 1
- ▶ By the end of this unit you’ll *be* one of those people

The pipeline – a teaser

- ▶ A language implementation transforms source text step by step
- ▶ The three stages we build this semester:
Scanner → Parser → Interpreter
- ▶ Today, just the names. We build each in turn from Week 4.

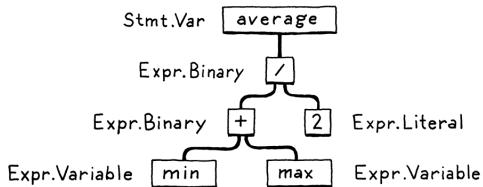
Source text becomes tokens

```
var average = (min + max) / 2;
```

`var` `average` `=` `(` `min` `+` `max` `)` `/` `2` `;`

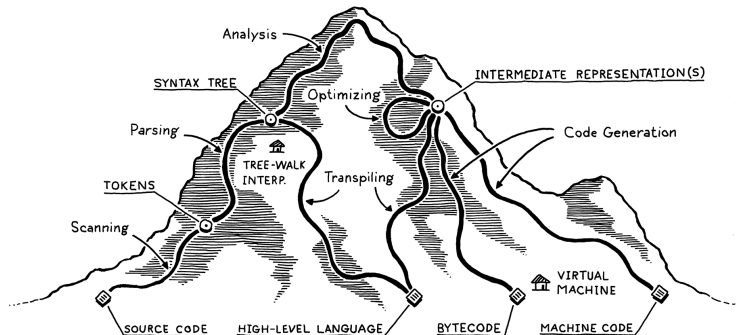
The scanner chunks characters into tokens.

Tokens become a tree



The parser builds an abstract syntax tree from the flat token stream.

The mountain, in full



- ▶ Not every implementation walks the whole mountain
- ▶ Our interpreter: **scan, parse, evaluate** – a tree-walk interpreter
- ▶ Later topics (Ch 11) add a static-analysis pass before evaluating

Wrap-up and Week 2 prep

- ▶ Today's reading: **Chapters 1–2** of *Crafting Interpreters*
- ▶ **Before next class:** read those chapters and complete the Week 2 iRAT
- ▶ Next week: little languages in more depth, and the *full* pipeline picture
- ▶ Reminder: form your team and write your team contract today

